OrbitalForge

A Seven-Day Modern C++ Scientific Engineering Bootcamp

Mission

Build a production-shaped orbital dynamics workbench in modern C++ while developing the language habits, debugging instincts, design judgment, and interview fluency expected from a strong intermediate C++ engineer.

Learning model: specification to backlog, failing test to implementation, debugger to explanation, benchmark to justified optimization. Every major concept is introduced by a real engineering pressure and reinforced through retrieval, repetition, and code review.

Complete execution curriculum • July 2026

Contents

> ***Mission:*** *Build a production-shaped orbital dynamics workbench in modern C++ while developing the language habits, debugging instincts, design judgment, and interview fluency expected from a strong intermediate C++ engineer.*

---

0. The Contract

This is not a seven-day tour of C++ syntax. It is a compressed engineering apprenticeship built around one scientific product. At the end of the week, you should be able to open an empty repository and independently structure, implement, test, debug, profile, and explain a medium-sized C++ application.

One week cannot make anyone an expert in the entire language. C++ is too large for that promise to be honest. The attainable target is sharper and more useful:

- You can write modern C++ without treating it as Python with semicolons.
- You understand object lifetime, ownership, values, references, pointers, and moves.
- You can split a program into headers, implementation files, libraries, executables, and tests.
- You can use the standard library deliberately rather than recreating it.
- You can recognize undefined behavior, iterator invalidation, dangling references, race conditions, and unnecessary allocation.
- You can discuss the complexity and memory behavior of your design.
- You can solve common interview tasks in idiomatic C++.
- You finish with a real scientific application that can grow after the bootcamp.

The weekly rule is simple:

> ***You type every important line. The compiler, tests, debugger, profiler, and I act as instructors.***

Do not paste complete implementations that you have not mentally reconstructed. A line that appears in the repository but not in your head is learning debt.

---

1. Product Mission

1.1 What OrbitalForge does

OrbitalForge is a command-line scientific application that simulates gravitational interaction among multiple bodies. It begins with a two-body orbit and grows into a reusable N-body laboratory with multiple numerical integrators, conservation diagnostics, scenario files, reproducible random systems, benchmark tools, parameter sweeps, and parallel execution.

By the end of Day 7, the application should support commands conceptually similar to:

```
orbitalforge simulate scenarios/two_body.orbit \
  --integrator verlet \
  --dt 0.001 \
  --steps 100000 \
  --output runs/two_body

orbitalforge compare scenarios/three_body_figure_eight.orbit \
  --integrators euler,verlet,rk4 \
  --metric energy-drift

orbitalforge benchmark scenarios/random_cluster_1000.orbit \
  --threads 1,2,4,8

orbitalforge sweep scenarios/two_body.orbit \
  --parameter dt \
  --values 0.1,0.05,0.025,0.0125 \
  --threads 4
```

A completed run produces:

```
runs/two_body/
├── metadata.txt
├── trajectory.csv
├── diagnostics.csv
└── summary.txt
```

The summary should report:

- scenario name;
- number of bodies;
- integrator;
- time step;
- number of steps;
- random seed if applicable;
- elapsed wall-clock time;
- initial and final energy;
- relative energy drift;
- momentum drift;
- center-of-mass drift;
- output paths;
- whether all scientific validity checks passed.

1.2 Scientific model

For bodies $i = 1, \dots, N$, each body has mass m_i, position $\mathbf{x}_i$, and velocity $\mathbf{v}_i$. The gravitational acceleration is:

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{x}_j - \mathbf{x}_i}{\left(\|\mathbf{x}_j - \mathbf{x}_i\|^2 + \epsilon^2 \right)^{3/2}}$$

where:

- G is the gravitational constant in the selected unit system;
- ϵ is an optional softening parameter used to avoid singular numerical behavior in very close encounters;
- the direct algorithm has $O(N^2)$ force-computation cost per step.

The simulator tracks several physical invariants:

$$E = \sum_i \frac{1}{2} m_i \|\mathbf{v}_i\|^2 - G \sum_{i<j} \frac{m_i m_j}{\|\mathbf{x}_j - \mathbf{x}_i\|}$$

$$\mathbf{P} = \sum_i m_i \mathbf{v}_i$$

$$\mathbf{X}_{cm} = \frac{\sum_i m_i \mathbf{x}_i}{\sum_i m_i}$$

These quantities give the project something many learning projects lack: objective signals that your code is scientifically wrong even when it compiles.

1.3 Required scenarios

The repository must eventually contain at least these scenarios:

1. **Static single body**
   No forces, no motion, perfect baseline for state and I/O tests.
2. **Two-body circular orbit**
   Primary integrator validation case.
3. **Two equal masses orbiting their center of mass**
   Tests symmetry and center-of-mass conservation.
4. **Three-body figure-eight orbit**
   Visually interesting and numerically sensitive.
5. **Sun-Earth-Moon inspired system**
   Uses scaled or normalized units to avoid unnecessary unit complexity during the week.
6. **Random cluster with a fixed seed**
   Used for reproducibility, performance measurements, and concurrency.

---

2. Why This Project Is the Right Learning Vehicle

2.1 It teaches C++ through recurring pressure

A good learning project does not merely permit a language feature. It creates pressure that makes the feature necessary.

Project pressure	C++ concept it naturally demands
Represent 3D mathematical quantities	value types, templates, operators, `constexpr`, const correctness
Store an unknown number of bodies	`std::vector`, contiguous storage, iterators, references
Expose body data without transferring ownership	`std::span`, const views, raw pointers as non-owning addresses
Choose an integration algorithm at runtime	interfaces, abstract classes, virtual functions, `override`, `unique_ptr`
Avoid leaks and cleanup bugs	RAII, rule of zero, destructors, smart pointers
Parse scenario text	strings, `string_view`, `from_chars`, optionals, variants, error handling
Export scientific results	streams, filesystem, formatting, exceptions
Validate numerical correctness	unit tests, approximate comparison, property tests, assertions
Compare algorithms	polymorphism, factories, enums, algorithms, ranges
Increase body count	complexity, profiling, cache locality, allocation behavior
Run independent experiments concurrently	`jthread`, atomics, mutexes, condition variables, race analysis
Optimize without changing behavior	benchmarks, data layout, compiler optimization, sanitizers

2.2 Why not a game, GUI, or generic data-structure library

A game would bury the learning under rendering, window systems, asset loading, and framework callbacks. A GUI application would create similar noise. A custom container library would teach low-level mechanics, but it would not naturally exercise scientific validation, I/O, runtime strategy selection, concurrency, or end-to-end product design.

OrbitalForge keeps the domain stable while the engineering depth rises. Every day improves the same application. That continuity is essential for fast learning because working memory is spent on new C++ ideas rather than constantly learning a new domain.

---

3. Final Product Scope

3.1 Core scope, mandatory

The following must work before the week is considered complete:

- C++20 project built with CMake.
- Library target, CLI target, and test target.
- Header and source separation.
- `Vec3<T>` value type.
- `Body`, `SystemState`, `SimulationConfig`, and diagnostics types.
- Direct $O(N^2)$ gravity computation.
- Euler and velocity-Verlet integrators.
- RK4 or leapfrog as a third integrator.
- Runtime integrator selection.
- Scenario parsing from a text file.
- CSV trajectory and diagnostics output.
- Energy, momentum, and center-of-mass diagnostics.
- Reproducible random scenario generation.
- Unit, property, integration, and regression tests.
- AddressSanitizer and UndefinedBehaviorSanitizer clean run.
- Benchmark command.
- Parallel parameter sweep or parallel force computation.
- ThreadSanitizer clean run for concurrent code.
- README with build, run, architecture, and scientific notes.
- A five-minute verbal explanation of ownership and architecture.

3.2 Stretch scope, only after core completion

Choose in this order:

1. Barnes-Hut quadtree or octree approximation.
2. Structure-of-arrays body storage and benchmark against array-of-structures.
3. `std::pmr` memory resource experiment.
4. SIMD-friendly force kernel.
5. C++23 `std::expected` error pipeline.
6. Plugin-like integrator registration.
7. Binary checkpoint serialization.
8. Live visualization through a minimal external plotting script.
9. Python bindings with pybind11.
10. Expression templates for vector arithmetic, strictly as an advanced experiment.

The stretch list is a ladder, not a buffet. Do not touch item 4 while core tests are failing.

---

4. Engineering Standards for the Week

4.1 Language baseline

Use **C++20**. This gives access to:

- concepts;
- ranges;
- `std::span`;
- `std::jthread`;
- `std::stop_token`;
- designated initializers for aggregates;
- improved `constexpr`;
- `std::source_location`.

C++23 features may appear only in isolated stretch exercises. Interview readiness requires being comfortable in C++17 and C++20 first.

4.2 Compiler settings

Use both GCC and Clang if available. The normal development build should be strict:

```
-Wall
-Wextra
-Wpedantic
-Wconversion
-Wsign-conversion
-Wshadow
-Wnon-virtual-dtor
-Wold-style-cast
-Woverloaded-virtual
-Wnull-dereference
-Wdouble-promotion
-Wformat=2
```

Some warnings may be noisy. Do not disable one before understanding why it fired.

Required build modes:

```
Debug
Release
ASan+UBSan
TSan
```

4.3 Repository structure

```
orbitalforge/
├── CMakeLists.txt
├── cmake/
│   └── CompilerWarnings.cmake
```

```
├── include/
│   └── orbitalforge/
│       ├── math/
│       │   └── vec3.hpp
│       ├── model/
│       │   ├── body.hpp
│       │   ├── system_state.hpp
│       │   └── units.hpp
│       ├── physics/
│       │   ├── gravity.hpp
│       │   └── diagnostics.hpp
│       ├── integration/
│       │   ├── integrator.hpp
│       │   ├── euler.hpp
│       │   ├── velocity_verlet.hpp
│       │   └── rk4.hpp
│       ├── io/
│       │   ├── scenario_parser.hpp
│       │   ├── csv_writer.hpp
│       │   └── run_directory.hpp
│       ├── app/
│       │   ├── simulation.hpp
│       │   ├── commands.hpp
│       │   └── benchmark.hpp
│       └── util/
│           ├── timer.hpp
│           ├── parse.hpp
│           └── assertions.hpp
├── src/
│   ├── physics/
│   ├── integration/
│   ├── io/
│   ├── app/
│   └── main.cpp
├── tests/
│   ├── unit/
│   ├── property/
│   ├── integration/
│   └── regression/
├── scenarios/
├── scripts/
├── docs/
│   ├── architecture.md
│   ├── numerical_methods.md
│   ├── ownership.md
│   └── learning_log.md
└── runs/
```

Do not create every file on Day 1. The structure is the destination. Files appear when a requirement makes them useful.

4.4 Target structure

Use at least these CMake targets:

```
orbitalforge_core      static or object library
orbitalforge           CLI executable
orbitalforge_tests     test executable
```

The CLI must link to the library. Tests must also link to the library. This prevents business logic from becoming trapped inside `main.cpp`.

4.5 Definition of Done

A task is done only when all applicable boxes are checked:

☐ behavior is implemented;
☐ code compiles with strict warnings;
☐ unit or integration tests exist;
☐ tests fail when the implementation is deliberately broken;
☐ names communicate intent;
☐ ownership is obvious from the interface;
☐ no unexplained raw owning pointer exists;
☐ complexity is known;
☐ no debug print remains;
☐ sanitizer run is clean;
☐ learning log contains the concept in your own words;
☐ Git commit describes why, not merely what.

---

5. The Learning Engine

5.1 Daily learning loop

Every concept goes through five passes:

1. **Prediction**
   Before compiling, state what you expect the code to do.
2. **Construction**
   Write the smallest implementation that makes one test pass.
3. **Breakage**
   Intentionally create a bug related to the concept.
4. **Diagnosis**
   Use compiler output, tests, debugger, or sanitizer to find it.
5. **Explanation**
   Explain the rule without reading notes, then repair the code.

This converts passive recognition into retrieval strength.

5.2 The three-pass implementation rule

Every meaningful feature is implemented in three passes:

Pass A: Correctness

Use the clearest simple solution. Explicit loops are allowed. Avoid premature abstraction.

Pass B: Design

Improve interfaces, const correctness, ownership, naming, file boundaries, and tests.

Pass C: Performance

Measure first. Then reduce allocation, copies, cache misses, or algorithmic cost.

Skipping directly to Pass C creates fast wrong code, a tiny race car with square wheels.

5.3 Spaced repetition schedule

Concepts are revisited on a fixed rhythm:

- **Same day:** five-minute recall after implementation.
- **Next morning:** reimplement a miniature form from memory.
- **Three days later:** explain and modify it in another subsystem.
- **Day 7:** solve an interview-style variant without project notes.

Examples:

- References are learned in `Vec3`, revisited in force computation, then revisited in parser and thread APIs.

- RAII is learned through file streams and timers, revisited through smart pointers and locks.
- Move semantics are learned through instrumented value types, revisited through vectors, factory returns, threads, and result aggregation.
- Const correctness begins in `Vec3`, then spreads across every public interface.

5.4 Retrieval notebook

Maintain `docs/learning_log.md`. Every day, answer these from memory:

```
## Concept
### My definition
### The bug it prevents
### A project example
### A counterexample
### One interview question
### What still confuses me
```

Do not copy a textbook definition. The notebook is a map of your own brain, potholes included.

5.5 Error ledger

Maintain a table:

Error	Root cause	Tool that exposed it	General rule	Prevention

Examples to collect deliberately:

- linker error from a missing definition;
- multiple-definition error from a non-inline function in a header;
- dangling `string_view`;
- vector iterator invalidation;
- unsigned underflow;
- slicing through pass-by-value base class;
- missing virtual destructor;
- use-after-move assumption;
- data race;
- false benchmark caused by compiler optimization.

Your mistakes become a private C++ interview bank.

---

6. Collaboration Protocol With the Assistant

To ensure you actually learn to code, use this escalation ladder:

1. State the behavior you want.
2. Sketch the types and function signature yourself.
3. Write a failing test.
4. Attempt the implementation for at least 15 focused minutes.
5. Share the smallest failing snippet and exact compiler/test output.
6. Ask first for a diagnosis or hint, not a full replacement file.
7. After repairing it, explain the cause back in your own words.
8. Reimplement the same pattern once without looking.

The assistant should provide complete code only when:

- the purpose is to compare against your finished attempt;
- the code is infrastructure unrelated to the current learning target;
- you are blocked by an external tool rather than C++;
- the day would otherwise collapse from a non-pedagogical issue.

A useful prompt pattern is:

```
I expected X, observed Y, and believe the cause is Z.
Here is the smallest code and the exact error.
Do not rewrite the file. Ask me questions or give one hint at a time.
```

---

7. Agile Project Management

7.1 One-week sprint goal

> *Deliver a scientifically validated, tested, benchmarked C++20 orbital simulation CLI whose architecture and ownership model you can defend in an interview.*

7.2 Product backlog

Epic A: Mathematical foundation

- A1: Implement a reusable three-dimensional vector value type.
- A2: Represent bodies and simulation state.
- A3: Implement direct gravitational acceleration.
- A4: Compute conservation diagnostics.

Epic B: Numerical integration

- B1: Euler integrator.
- B2: Velocity-Verlet integrator.
- B3: Third integrator.
- B4: Integrator selection and comparison.

Epic C: Application usability

- C1: Scenario file format.
- C2: Parser and validation.
- C3: CLI commands.
- C4: Run directory and CSV output.
- C5: Reproducible random scenarios.

Epic D: Quality

- D1: Unit tests.
- D2: Property tests.
- D3: Integration tests.
- D4: Regression scenarios.
- D5: Sanitizer builds.
- D6: Documentation.

Epic E: Performance and concurrency

- E1: Benchmark harness.
- E2: Complexity experiment.
- E3: Parallel parameter sweep.
- E4: Parallel force kernel or data-layout experiment.
- E5: ThreadSanitizer validation.

7.3 Daily board

Use only four columns:

```
BACKLOG | TODAY | VERIFY | DONE
```

WIP limit:

- at most one coding story in `TODAY`;
- at most two tasks in `VERIFY`;
- no new feature while a core test is red.

7.4 Story template

```
## Story
As a ...
I want ...
So that ...

### Acceptance criteria
- Given ...
- When ...
- Then ...

### Learning target
The C++ concept this story forces me to practice is ...

### Test first
The first failing test will be ...

### Risks
- Numerical:
- Ownership:
- Complexity:
```

7.5 Git discipline

Create small commits with messages such as:

```
build: create core library and test target
test(vec3): define arithmetic and tolerance behavior
feat(gravity): add symmetric direct-force accumulation
refactor(integration): extract runtime integrator interface
fix(parser): prevent dangling string_view into temporary line
perf(force): parallelize independent acceleration rows
docs(ownership): record observer and owner rules
```

End each day with a tag:

```
day-1-foundation
day-2-gravity
day-3-integrators
day-4-application
day-5-modern-cpp
day-6-performance
day-7-release
```

---

8. Topic Coverage Map

The purpose of this table is not to chase checkmarks. It proves that each important topic has a scheduled home.

Topic	Primary day	Project location
Compilation pipeline, preprocessing, linking	1	build and translation-unit lab
Headers, include guards, ODR, linkage	1	`vec3.hpp`, `body.hpp`, source files
Namespaces	1	all library code
Fundamental types and conversions	1	model and numeric validation
Initialization and narrowing	1	all value construction
References and const	1-2	vector operators and physics kernels
Raw pointers	2	address/observer lab and C interop thought exercise
`std::array`, `std::vector`, `std::span`	1-2	vectors, body storage, kernel views
Struct versus class	1	`Body` versus `Vec3`
Constructors and invariants	1	`Vec3`, config
Operator overloading	1	`Vec3`
Templates	1, 5	`Vec3<T>`, generic algorithms
Concepts	5	numeric and integrator constraints
Iterators and algorithms	2, 5	body processing and diagnostics
Lambdas	2, 5, 6	transforms, validation, threads
RAII	1-7	streams, timer, pointers, locks, threads
Copy, move, value categories	3	instrumented buffer and factories
Rule of zero/five	3	value types and owned strategies
Abstract classes and virtual functions	3	integrator strategy
`unique_ptr`, `shared_ptr`, `weak_ptr`	3	runtime ownership labs
Exceptions and assertions	4	parser and invariant boundaries
`optional`, `variant`	4	parsing and command/config representation
Strings and `string_view`	4	parser
Streams, filesystem, serialization	4	run output
`chrono`	4, 6	metadata and benchmarks

Topic	Primary day	Project location
Random library	4	generated scenarios
Ranges	5	analysis and reports
`constexpr`, `consteval`, type traits	5	math and compile-time checks
Memory layout, alignment, cache	6	AoS/SoA benchmark
Threads, mutexes, atomics	6	sweep and parallel kernel
Testing, debugger, sanitizers	every day	all subsystems
Profiling and optimization	6-7	benchmarks
Design patterns	3-4	Strategy and Factory only where justified
Interview explanation	every day	verbal checkpoints

9. Daily Operating Schedule

Each day assumes approximately **8 focused hours** plus breaks. The blocks are deliberately repetitive. Consistent ritual reduces decision fatigue.

Time block	Activity
00:00-00:25	Closed-book retrieval from previous days
00:25-01:10	Concept lesson and tiny experiments
01:10-02:40	Test-first implementation block A
02:40-03:00	Break and verbal explanation
03:00-04:30	Implementation block B
04:30-05:10	Lunch or long break
05:10-06:10	Debugging or deliberate bug laboratory
06:10-07:10	Refactor, documentation, and strict build
07:10-07:40	Interview drills
07:40-08:00	Retrospective, commit, and tomorrow setup

Rules:

- During implementation blocks, use 50 minutes of work followed by 10 minutes away from the screen.
- At the end of each 50-minute cycle, write one sentence stating what changed in your mental model.
- Stop adding features 70 minutes before the end of the day.
- The final 70 minutes are reserved for tests, cleanup, explanation, and a stable commit.

---

10. Day 1: Build System, Compilation Model, and Value Types

10.1 Daily mission

Create a professional C++ repository that builds a library, executable, and test target. Implement the mathematical value type and body model without any gravitational simulation yet.

At the end of Day 1, this must work:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build
ctest --test-dir build --output-on-failure
./build/orbitalforge
```

The executable prints a deterministic summary of two constructed bodies and several `Vec3` operations.

10.2 Concepts

Compilation model

Learn the path:

```
source file
-> preprocessing
-> compilation
-> object file
-> linking
-> executable
```

You must understand:

- `#include` performs textual inclusion;
- declarations tell the compiler that something exists;
- definitions provide storage or implementation;
- every `.cpp` becomes a translation unit after preprocessing;
- the linker combines object files;
- templates usually need definitions visible at the point of instantiation;
- non-inline function definitions in headers can violate the One Definition Rule;
- `inline` is mainly an ODR/linkage tool, not a command to the optimizer;
- anonymous namespaces and `static` at namespace scope provide internal linkage;
- include guards or `#pragma once` prevent repeated inclusion within a translation unit.

Fundamental types

Cover:

- `bool`;
- `char`, signed and unsigned character types;
- fixed-width integers such as `std::int32_t` and `std::uint64_t`;

- `int`, `long`, `long long`;
- `std::size_t`, `std::ptrdiff_t`;
- `float`, `double`, `long double`;
- `std::nullptr_t`;
- `auto` and `decltype`;
- signed/unsigned comparison hazards;
- narrowing conversion;
- overflow and underflow;
- floating-point approximation;
- `std::numeric_limits`.

Project rules:

- `double` is the main physical scalar.
- `std::size_t` represents container size and indices.
- A fixed-width integer stores reproducible seeds.
- Never use an integer sentinel such as `-1` in an unsigned type.
- Use braces for initialization unless a specific constructor syntax is required.
- Do not compare floating-point values with exact equality in scientific tests.

Value types and class design

Learn:

- aggregate `struct`;
- invariant-owning `class`;
- public/private;
- constructors;
- member initialization lists;
- `explicit`;
- `const` member functions;
- `static` members;
- free functions versus member functions;
- operators;
- rule of zero;
- `[[nodiscard]]`;
- `constexpr`;
- `noexcept` for obviously non-throwing tiny operations.

10.3 Files created

```
CMakeLists.txt
cmake/CompilerWarnings.cmake
include/orbitalforge/math/vec3.hpp
include/orbitalforge/model/body.hpp
src/main.cpp
tests/unit/test_vec3.cpp
docs/architecture.md
docs/learning_log.md
```

10.4 Exact implementation sequence

Task 1: Toolchain fingerprint

Make the executable print:

- compiler family and version if available through macros;
- C++ language version;
- `sizeof(int)`, `sizeof(std::size_t)`, `sizeof(double)`, and `sizeof(void*)`;
- maximum `double`;
- machine epsilon for `double`.

Purpose: remove the assumption that type sizes and numeric properties are magical constants.

Task 2: Translation-unit laboratory

Create:

```
include/orbitalforge/demo.hpp
src/demo.cpp
src/main.cpp
```

Perform these experiments one at a time:

1. Declare a function in a header and define it in a source file.
2. Remove the definition and observe the linker error.
3. Define a normal function in the header, include it in two sources, and observe the multiple-definition failure.
4. Mark the header-defined function `inline` and explain why linking succeeds.
5. Put a helper function in an anonymous namespace in `demo.cpp`.
6. Use the preprocessor output option (`-E`) and inspect the expanded file.

Record each error in the error ledger.

Delete the demo after the lesson or keep it under `experiments/day1/`.

Task 3: Implement `Vec3<T>`

Required behavior:

```
Vec3<double> a{1.0, 2.0, 3.0};
Vec3<double> b{4.0, 5.0, 6.0};

auto c = a + b;
auto d = 2.0 * a;
auto dot = dot(a, b);
auto norm = a.norm();
```

Required operations:

- default construction to zero;
- three-value construction;

- `x()`, `y()`, `z()` accessors;
- mutable and const accessors, or carefully chosen public data if you can defend it;
- `operator+=`, `operator-=`, `operator*=`, `operator/=`;
- non-member `+`, `-`, scalar multiplication, scalar division;
- dot product;
- squared norm;
- norm;
- equality only if its semantics are explicitly defined;
- `almost_equal` free function for tests;
- stream output;
- optional cross product.

Design questions to answer:

- Why is `operator+` implemented in terms of `operator+=`?
- Why should scalar multiplication support both `v * s` and `s * v`?
- Which operations can be `constexpr`?
- Should `norm()` be `noexcept`?
- Why is a free `dot(a,b)` often preferable to `a.dot(b)`?
- Why should a converting constructor normally be `explicit`?

Task 4: Implement Body

Use a simple aggregate initially:

```
struct Body {
    std::string name;
    double mass;
    Vec3<double> position;
    Vec3<double> velocity;
};
```

Use designated initialization:

```
Body earth{
    .name = "Earth",
    .mass = 1.0,
    .position = {1.0, 0.0, 0.0},
    .velocity = {0.0, 1.0, 0.0},
};
```

Write a validation function rather than hiding premature complexity inside setters.

Questions:

- Why is `Body` a `struct` while `Vec3` may be a `class`?
- Which invariants belong inside the type?
- Is negative mass representable, and where should it be rejected?
- Is a body name required to be unique?

10.5 Tests

Required tests:

- default vector is zero;
- vector addition and subtraction;
- scalar multiplication in both orders;
- compound assignment;
- dot product;
- norm of `{3,4,0}`;
- approximate equality;
- const access works;
- body aggregate construction;
- negative mass validation fails.

Use a test framework through CMake `FetchContent`, preferably Catch2 or GoogleTest. Do not spend more than 45 minutes debating frameworks.

10.6 Deliberate bug lab

Create and diagnose:

1. A narrowing initialization:

```
int steps{10.5};
```

2. Signed/unsigned loop bug.
3. Missing function definition.
4. Multiple definition from a header.
5. Uninitialized local scalar.
6. Exact floating-point equality failure.
7. Integer division accidentally assigned to `double`.

10.7 Interview drills

Answer aloud without notes:

1. What is a translation unit?
2. What is the difference between a declaration and a definition?
3. Why are template definitions commonly in headers?
4. What is the One Definition Rule?
5. `struct` versus `class`?
6. Why use member initializer lists?
7. What does `explicit` prevent?
8. What does `const` after a member function mean?
9. Why is `std::size_t` dangerous in subtraction?
10. What is the difference between `float`, `double`, and `long double`?
11. What does `constexpr` guarantee?
12. Is `inline` a performance command?

10.8 Acceptance gate

☐ three targets build;
☐ tests pass;
☐ strict warnings are enabled;
☐ `Vec3<T>` is usable as a normal value;
☐ no manual allocation exists;
☐ translation-unit errors are documented;
☐ you can explain every line of the top-level CMake file;
☐ tag `day-1-foundation` exists.

---

11. Day 2: Containers, References, Iterators, and Gravity

11.1 Daily mission

Implement the direct gravitational acceleration kernel and physical diagnostics for a collection of bodies.

At the end of the day:

```
orbitalforge inspect scenarios/two_body.orbit
```

may still use a hardcoded scenario internally, but it reports masses, accelerations, total momentum, center of mass, kinetic energy, and potential energy.

11.2 Morning retrieval

Without opening the repository:

1. Recreate a minimal `Vec2<T>` on paper.
2. Explain declaration, definition, translation unit, and linker.
3. Write a function signature accepting a read-only sequence of bodies without ownership.
4. Predict what happens when a `std::vector` grows beyond capacity.

11.3 Concepts

Ownership vocabulary

Use these words precisely:

- **owner:** responsible for object lifetime;
- **observer:** accesses an object but does not control lifetime;
- **borrow:** temporary non-owning access;
- **value:** independent object with its own lifetime;
- **reference:** non-null alias that cannot be reseated;
- **pointer:** nullable address-like value that can be reseated;
- **span:** non-owning view over contiguous elements.

Project ownership policy:

```
std::vector<Body>       owns bodies
std::span<const Body>   borrows bodies read-only
std::span<Vec3d>        borrows writable accelerations
Body&                   borrows one required body
const Body&             borrows one required read-only body
Body*                    observes one optional body
```

A raw pointer does not automatically mean bad code. An unexplained owning raw pointer is the problem.

References

Cover:

- lvalue reference `T&`;
- const reference `const T&`;
- reference binding;
- reference lifetime;
- dangling references;
- pass by value versus pass by reference;
- return by reference;
- `std::reference_wrapper`;
- why a vector cannot store references directly.

Containers

Main project:

- `std::array` for fixed-size mathematical storage if chosen inside `Vec3`;
- `std::vector` for bodies and accelerations;
- `std::span` for kernel interfaces.

Micro-lab comparison:

- `std::deque`;
- `std::list`;
- `std::map`;
- `std::unordered_map`;
- `std::set`;
- `std::unordered_set`;
- `std::stack`;
- `std::queue`;
- `std::priority_queue`.

You do not need to force all of them into OrbitalForge. You must know their ownership, ordering, iterator invalidation, complexity, and memory-locality tradeoffs.

Iterators and invalidation

Cover:

- `begin`, `end`;
- iterator categories conceptually;
- range-for desugaring;
- iterator and reference invalidation after vector reallocation;
- `reserve` versus `resize`;
- `push_back` versus `emplace_back`;
- index loop versus iterator loop versus range-for;
- when algorithms communicate intent better than loops.

11.4 Files created

```
include/orbitalforge/model/system_state.hpp
include/orbitalforge/physics/gravity.hpp
include/orbitalforge/physics/diagnostics.hpp
src/physics/gravity.cpp
src/physics/diagnostics.cpp
tests/unit/test_gravity.cpp
tests/unit/test_diagnostics.cpp
tests/property/test_physics_properties.cpp
```

11.5 Data model

Begin with:

```cpp
class SystemState {
public:
    explicit SystemState(std::vector<Body> bodies);

    [[nodiscard]] std::span<const Body> bodies() const noexcept;
    [[nodiscard]] std::span<Body> bodies() noexcept;
    [[nodiscard]] std::size_t size() const noexcept;

private:
    std::vector<Body> bodies_;
};
```

Questions:

- Should `SystemState` expose writable spans?
- Could writable access break invariants?
- Should it own accelerations too?
- Is construction by value efficient?
- What happens when the vector argument is moved into the member?

Do not blindly make everything a class. The point is to debate invariants.

11.6 Gravity kernel, staged implementation

Stage A: Clear $O(N^2)$ version

Create an output vector initialized to zero:

```cpp
std::vector<Vec3d> accelerations(bodies.size());
```

For every `i`, iterate over every `j != i`. Write only to `accelerations[i]`.

Interface:

```
void compute_accelerations(
    std::span<const Body> bodies,
    std::span<Vec3d> accelerations,
    double gravitational_constant,
    double softening);
```

Validation:

- output span has same size as body span;
- masses are positive;
- softening is non-negative;
- positions and masses are finite.

Stage B: Symmetric pair accumulation

Refactor to iterate only over `i < j`. Use Newton's third-law symmetry to update both bodies.

Discuss:

- fewer pair evaluations;
- two writes per pair;
- how this complicates parallelization later;
- whether numerical rounding changes;
- why tests should validate behavior rather than exact implementation.

Stage C: Allocation-free hot function

The kernel must not allocate. The caller owns acceleration storage. This is a first encounter with performance-shaped API design.

11.7 Diagnostics

Implement:

```
double kinetic_energy(std::span<const Body>);
double potential_energy(std::span<const Body>, double G, double softening);
Vec3d total_momentum(std::span<const Body>);
Vec3d center_of_mass(std::span<const Body>);
```

Use `std::accumulate` or explicit loops first, then compare clarity.

11.8 Tests

Deterministic tests:

- one body has zero acceleration;
- two equal masses experience equal and opposite forces;
- acceleration points toward the other body;

- doubling the attracting mass doubles acceleration;
- doubling distance reduces magnitude by approximately four;
- center of mass for symmetric pair is origin;
- momentum of equal opposite velocities is zero;
- potential energy is negative;
- invalid output span size is rejected.

Property tests with fixed random seed:

- sum of internal forces is approximately zero;
- translating every position by the same vector does not change pair accelerations;
- permuting body order permutes outputs but does not change physical values;
- total mass is positive for validated state.

11.9 Container micro-lab

Write a 60-line experimental program that:

1. Inserts 100,000 integers into `vector`, `deque`, and `list`.
2. Traverses and sums them.
3. Times traversal.
4. Prints object addresses for several neighboring elements.
5. Explains why contiguous storage matters.

This is not a formal benchmark. It is a memory-layout intuition exercise.

Then implement a body-name lookup with `unordered_map<std::string,std::size_t>` and answer:

- why map values are indices rather than references;
- what vector reallocation does to pointers and references;
- what vector reordering does to stored indices;
- whether the lookup belongs inside `SystemState`.

11.10 Deliberate bug lab

1. Keep a reference to `vector[0]`, push enough elements to reallocate, then access the reference under ASan.
2. Use `reserve` and explain why the bug may disappear without becoming conceptually safe forever.
3. Return a reference to a local object.
4. Create a `span` to a temporary vector.
5. Use `vector<bool>` and inspect why its element access is unusual.
6. Erase from a vector while iterating incorrectly.

11.11 Interview drills

1. `vector` versus `array`.
2. `vector` versus `deque`.

3. Why is `list` often slower despite constant-time insertion?
4. `reserve` versus `resize`.
5. What invalidates vector iterators?
6. Pass by value versus `const&`.
7. Pointer versus reference.
8. What is `span`, and who owns the data?
9. Why not return a reference to a local?
10. Complexity of direct N-body acceleration.
11. Why can cache locality dominate asymptotic similarity?
12. `push_back` versus `emplace_back`.

11.12 Acceptance gate

☐ gravity tests pass;
☐ properties pass with deterministic seed;
☐ hot kernel performs no allocations;
☐ spans and references have documented lifetimes;
☐ vector invalidation bug is reproduced under sanitizer;
☐ physical complexity is explained;
☐ tag `day-2-gravity` exists.

---

12. Day 3: RAII, Copy/Move Semantics, Smart Pointers, and Polymorphism

12.1 Daily mission

Create the simulation stepping architecture and implement at least Euler and velocity-Verlet integrators. Select the integrator at runtime through a justified strategy interface.

12.2 Morning retrieval

From memory:

- write the gravity kernel signature;
- explain why it does not own its spans;
- list four causes of vector invalidation;
- derive the two-body equal-and-opposite force property;
- write a function that takes ownership of a vector efficiently.

12.3 Concepts

Object lifetime

Understand:

- automatic storage duration;
- static storage duration;
- dynamic storage duration;
- construction and destruction order;
- scopes;
- temporaries;
- lifetime extension of temporaries;
- object slicing;
- base and derived destruction.

RAII

RAII means resource acquisition is bound to object lifetime. Resources include:

- heap memory;
- files;
- locks;
- threads;
- sockets;
- timers or tracing scopes;
- temporary directories.

The destructor provides deterministic cleanup. Exceptions do not bypass it.

Special member functions

Learn:

- destructor;
- copy constructor;
- copy assignment;
- move constructor;
- move assignment;
- rule of three;
- rule of five;
- rule of zero;
- deleted functions;
- defaulted functions;
- moved-from state;
- `std::move` as a cast that permits moving;
- return-value optimization;
- copy elision;
- `noexcept` move and container behavior.

Smart pointers

Use:

- `std::unique_ptr<T>` for exclusive ownership;
- `std::shared_ptr<T>` only for genuine shared lifetime;
- `std::weak_ptr<T>` to observe shared ownership without extending lifetime or creating cycles;
- `make_unique` and `make_shared`;
- `.get()` only for non-owning interoperability;
- never call `delete` on `.get()`.

In the core architecture, the chosen integrator is owned by `unique_ptr`. There should be no need for `shared_ptr` in normal simulation code. Learn it in a controlled lab rather than infecting the design with shared ownership.

Runtime polymorphism

Learn:

- abstract base class;
- pure virtual function;
- virtual destructor;
- `override`;
- `final`;
- vtable concept;
- dynamic dispatch;
- object slicing;
- interface segregation;
- runtime cost and design cost.

Use inheritance only for substitutable behavior. A body is not a kind of vector. An integrator is a strategy.

12.4 Files created

```
include/orbitalforge/integration/integrator.hpp
include/orbitalforge/integration/euler.hpp
include/orbitalforge/integration/velocity_verlet.hpp
src/integration/euler.cpp
src/integration/velocity_verlet.cpp
include/orbitalforge/app/simulation.hpp
src/app/simulation.cpp
tests/unit/test_euler.cpp
tests/unit/test_velocity_verlet.cpp
tests/integration/test_two_body_orbit.cpp
experiments/day3/move_probe.cpp
experiments/day3/shared_cycle.cpp
```

12.5 Integrator interface

Start with a minimal interface:

```
class Integrator {
public:
    virtual ~Integrator() = default;

    virtual void step(
        SystemState& state,
        double dt,
        const SimulationParameters& parameters) = 0;

    [[nodiscard]] virtual std::string_view name() const noexcept = 0;
};
```

Design review:

- Why is the destructor virtual?
- Why does `step` take `SystemState&`?
- Why is `name()` read-only and non-throwing?
- Should `dt` be a raw `double` or a strong duration type?
- Does the integrator need to own temporary acceleration buffers?
- If it owns buffers, what happens when the number of bodies changes?

12.6 Factory

Use a scoped enum:

```cpp
enum class IntegratorKind {
    euler,
    velocity_verlet,
    rk4
};
```

Factory:

```cpp
std::unique_ptr<Integrator> make_integrator(IntegratorKind kind);
```

This naturally teaches returning move-only values. Do not write `std::move` on the local return unless you can explain why it may inhibit copy elision.

12.7 Euler implementation

Implement the simplest explicit method:

$$\mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \Delta t \mathbf{v}_t$$

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + \Delta t \mathbf{a}_t$$

Choose and document update ordering. Explicit Euler and semi-implicit Euler differ. Do not accidentally implement one while naming the other.

Use Euler as a pedagogical baseline, not the recommended production integrator.

12.8 Velocity-Verlet implementation

Implement:

1. compute acceleration a_t;
2. update position:
$$x_{t+\Delta t} = x_t + v_t \Delta t + \frac{1}{2} a_t \Delta t^2$$
3. compute new acceleration $a_{t+\Delta t}$;
4. update velocity:
$$v_{t+\Delta t} = v_t + \frac{1}{2}(a_t + a_{t+\Delta t})\Delta t$$

This requires reusable internal buffers. Keep them as vectors owned by the concrete integrator or allocate them in the simulation workspace. Compare both designs.

12.9 Move-semantics laboratory

Implement an `InstrumentedBuffer` owning `std::unique_ptr<double[]>` only as a learning experiment.

Manually implement:

- constructor;
- destructor;
- deleted or deep-copy operations;
- move constructor;
- move assignment;
- `swap`;
- `noexcept`.

Print lifecycle events. Then replace the raw array with `std::vector<double>` and delete all custom special members. This demonstrates why the rule of zero is the normal goal.

Questions:

- What does `std::move` actually do?
- What state must a moved-from object support?
- Why does a vector prefer a `noexcept` move during reallocation?
- Why is self-move assignment worth considering?
- Why is manual resource management educational but not the default architecture?

12.10 Shared ownership laboratory

Create two `shared_ptr<Node>` objects that point to each other and observe that destructors do not run. Replace one direction with `weak_ptr`. Explain:

- reference count;
- cycle;
- lock;
- expired;
- why shared ownership is semantic, not merely convenient.

Do not use shared ownership in OrbitalForge unless a later design truly needs it.

12.11 Tests

- factory returns correct integrator name;
- integrator pointer is non-null;
- one body preserves velocity and moves linearly;
- zero `dt` changes nothing;
- Euler performs expected one-step update;
- Verlet performs expected constant-acceleration update;
- two-body orbit remains bounded for a selected duration under Verlet;
- Verlet energy drift is lower than Euler under the same coarse step;
- destroying through base pointer is safe;
- copying `unique_ptr` fails at compile time in a small commented experiment.

12.12 Deliberate bug lab

1. Remove virtual destructor and compile with warning.
2. Pass a derived integrator by value to a function taking base, observe slicing.
3. Attempt to copy `unique_ptr`.

4. Dereference a moved-from `unique_ptr`.
5. Use `shared_ptr` cycle.
6. Throw during a scope holding a file or custom guard and verify cleanup.
7. Mark a throwing move constructor `noexcept`, then observe termination in a controlled test.

12.13 Interview drills

1. Rule of zero, three, and five.
2. What does `std::move` do?
3. Lvalue, xvalue, prvalue at an intuitive level.
4. Why is a moved-from object valid but unspecified?
5. `unique_ptr` versus `shared_ptr`.
6. Why use `weak_ptr`?
7. Why must a polymorphic base usually have a virtual destructor?
8. What is slicing?
9. Runtime polymorphism cost.
10. Factory returning `unique_ptr`.
11. RAII under exceptions.
12. Why can `noexcept` move matter to `vector`?

12.14 Acceptance gate

☐ two integrators work;
☐ runtime selection uses `unique_ptr`;
☐ no owning raw pointer exists in core code;
☐ Verlet outperforms Euler scientifically on the chosen test;
☐ move and shared-cycle labs are documented;
☐ ownership diagram exists in `docs/ownership.md`;
☐ tag `day-3-integrators` exists.

---

13. Day 4: Parsing, Error Handling, Filesystem, and Complete CLI Flow

13.1 Daily mission

Turn the numerical engine into a usable application. Read scenarios, validate them, run simulations, and export reproducible results.

13.2 Scenario format

Use a deliberately small line-based format so that parsing itself teaches C++:

```
# two_body.orbit
name = two-body-circular
gravitational_constant = 1.0
softening = 0.0
dt = 0.001
steps = 100000
sample_every = 10
integrator = velocity_verlet
seed = 42

body = primary,1.0,-0.5,0.0,0.0,0.0,-0.5,0.0
body = secondary,1.0,0.5,0.0,0.0,0.0,0.5,0.0
```

Do not build a universal parser. Build a correct parser for this grammar.

13.3 Concepts

Strings

Cover:

- `std::string`;
- `std::string_view`;
- ownership and lifetime;
- null termination;
- `char*` and C strings conceptually;
- substring;
- search;
- trimming;
- tokenization;
- `std::from_chars`;
- stream extraction;
- dangling views.

Error categories

Use different mechanisms for different meanings:

- programmer invariant violation: `assert`;
- invalid user input: parser error with line number;

- missing optional field: `std::optional`;
- one of several valid command forms: `std::variant`;
- unrecoverable file-open failure at application boundary: exception or explicit error object;
- impossible enum state: defensive failure.

Do not use exceptions for ordinary loop control. Do not return `nullptr` when an empty optional expresses the contract better.

Exceptions

Understand:

- `throw`;
- stack unwinding;
- catch by `const&`;
- standard exception hierarchy;
- exception safety;
- basic guarantee;
- strong guarantee;
- no-throw guarantee;
- destructor behavior;
- why exceptions should be translated near the application boundary.

Filesystem and streams

Cover:

- `std::filesystem::path`;
- directory creation;
- file existence;
- `ifstream`, `ofstream`;
- stream state;
- formatting precision;
- RAII closing;
- temporary output file and rename for safer writes;
- text versus binary.

Sum types

Use:

- `optional<T>` for possibly absent value;
- `variant<A,B,...>` for a value that is exactly one of several types;
- `visit` for variant processing;
- C++23 `expected<T,E>` as a stretch comparison.

13.4 Files created

```
include/orbitalforge/io/scenario_parser.hpp
src/io/scenario_parser.cpp
include/orbitalforge/io/csv_writer.hpp
src/io/csv_writer.cpp
include/orbitalforge/io/run_directory.hpp
src/io/run_directory.cpp
include/orbitalforge/app/commands.hpp
src/app/commands.cpp
include/orbitalforge/model/simulation_config.hpp
tests/unit/test_parser.cpp
tests/unit/test_csv_writer.cpp
tests/integration/test_cli_simulate.cpp
scenarios/two_body.orbit
scenarios/three_body_figure_eight.orbit
```

13.5 Parser design

Suggested result types:

```
struct ParseError {
    std::filesystem::path file;
    std::size_t line;
    std::string message;
};

struct Scenario {
    std::string name;
    SimulationConfig config;
    std::vector<Body> bodies;
};
```

C++20 choices:

- either throw a domain-specific `ScenarioParseException`;
- or return `std::variant<Scenario, ParseError>`;
- or use an output/error structure.

Choose one and defend it. A clean teaching option is to use a `variant` today and compare it to `expected` later.

Parsing pipeline:

1. read line;
2. preserve original line number;
3. remove comments;
4. trim whitespace;
5. skip empty line;
6. split on first =;
7. parse key;
8. dispatch value parser;

9. validate duplicates;
10. validate complete scenario after all lines are read.

Use a lookup table only if it improves clarity. A chain of explicit branches is acceptable for a small grammar.

13.6 Lifetime trap exercise

Write a function that returns `string_view` into a local string and demonstrate the bug. Then fix parser helpers so that views never outlive the original line.

Document this rule:

> *A view is safe only while the viewed storage remains alive and unmoved.*

13.7 CLI command model

Represent commands with a variant:

```
struct SimulateCommand { /* ... */ };
struct CompareCommand { /* ... */ };
struct BenchmarkCommand { /* ... */ };

using Command = std::variant<
    SimulateCommand,
    CompareCommand,
    BenchmarkCommand>;
```

Parse `argv` manually. This teaches:

- arrays of C-string pointers;
- pointer-to-pointer syntax at the program boundary;
- conversion to `span` or `string_view`;
- validation;
- variants;
- visitors.

Do not add a CLI dependency this week unless argument parsing begins consuming the day.

13.8 Output writer

Trajectory CSV:

```
step,time,body_id,name,x,y,z,vx,vy,vz
```

Diagnostics CSV:

```
step,time,kinetic,potential,total_energy,px,py,pz,cmx,cmy,cmz
```

Use sufficient precision:

```
stream << std::setprecision(std::numeric_limits<double>::max_digits10);
```

The writer should own its stream as a RAII resource. Decide whether it is movable and non-copyable.

13.9 Random scenarios

Use:

- `std::mt19937_64`;
- fixed-width seed;
- uniform or normal distributions;
- deterministic default seed written to metadata;
- optional random-device-generated seed only when explicitly requested.

Separate engine from distribution. Never use `std::rand`.

13.10 Tests

Parser tests:

- valid minimal file;
- comments and whitespace;
- missing required field;
- duplicate key;
- unknown key;
- malformed number;
- negative mass;
- duplicate body name;
- zero bodies;
- line number in error;
- deterministic parse result.

I/O tests:

- output directory created;
- files open;
- header correct;
- expected row count;
- floating-point round-trip within tolerance;
- run metadata includes seed and integrator.

CLI integration test:

- run a short scenario;
- exit code is zero;
- output files exist;
- summary contains PASS;
- malformed scenario returns non-zero and a useful message.

13.11 Deliberate bug lab

1. Dangling `string_view`.
2. Catch exception by value and explain slicing.
3. Throw from a destructor during unwinding in an isolated program.
4. Forget to test stream state.
5. Parse `-1` into an unsigned value.
6. Use `std::stod` without checking full-token consumption.
7. Write output directly to final file, crash midway, inspect corruption.

13.12 Interview drills

1. `string` versus `string_view`.
2. When does a view dangle?
3. Exceptions versus error codes.
4. `optional` versus pointer.
5. `variant` versus inheritance.
6. Why catch exceptions by `const&`?
7. What is stack unwinding?
8. Exception guarantees.
9. Why use `filesystem::path`?
10. How does RAII help file I/O?
11. `from_chars` versus stream parsing.
12. What is `argv` really?

13.13 Acceptance gate

☐ scenario file drives simulation;
☐ invalid input produces precise diagnostics;
☐ CSV output is reproducible;
☐ seed is recorded;
☐ no dangling view exists;
☐ CLI integration test passes;
☐ tag `day-4-application` exists.

---

14. Day 5: Templates, Concepts, Ranges, Compile-Time Work, and Generic Design

14.1 Daily mission

Modernize the code where generic programming genuinely improves reuse and correctness. Add the third integrator, comparison command, and analysis pipeline.

This is not “template everything” day. It is “learn where compile-time abstraction beats runtime abstraction” day.

14.2 Morning retrieval

Closed book:

- draw the ownership graph;
- write the parser result type;
- list three dangling-view patterns;
- explain `unique_ptr` factory return;
- derive the Verlet step;
- state the exception boundary.

14.3 Concepts

Templates

Cover:

- function templates;
- class templates;
- template argument deduction;
- non-type template parameters;
- template instantiation;
- dependent names at a basic level;
- specialization conceptually;
- alias templates;
- variadic templates at a controlled level;
- header visibility;
- code bloat tradeoff.

Concepts and constraints

Use:

```
template<std::floating_point T>
class Vec3;
```

Potential integrator concept:

```cpp
template<class T>
concept StepIntegrator =
    requires(T integrator, SystemState& state, double dt,
             const SimulationParameters& params) {
        { integrator.step(state, dt, params) } -> std::same_as<void>;
        { integrator.name() } -> std::convertible_to<std::string_view>;
    };
```

Compare:

- runtime polymorphism with virtual interface;
- compile-time polymorphism with constrained templates;
- `std::variant` static dispatch;
- type erasure conceptually.

`constexpr`

Cover:

- compile-time evaluability;
- literal types;
- constexpr constructors;
- compile-time vector arithmetic;
- `static_assert`;
- `constinit`;
- `consteval` as a small lab;
- compile-time versus runtime cost;
- why not every function needs `constexpr`.

Standard algorithms and ranges

Use:

- `transform`;
- `accumulate`;
- `reduce` conceptually;
- `sort`;
- `find_if`;
- `all_of`;
- `any_of`;
- `minmax_element`;
- ranges views;
- projections;
- lambdas;
- captures;
- generic lambdas.

Avoid turning clear numerical loops into cryptic pipelines. Ranges are a tool, not decorative plumbing.

Callable types

Understand:

- function pointer;
- functor;
- lambda;
- generic lambda;
- `std::function`;
- template callable parameter;
- allocation and type-erasure cost of `std::function`.

14.4 Files created

```
include/orbitalforge/integration/rk4.hpp
src/integration/rk4.cpp
include/orbitalforge/app/comparison.hpp
src/app/comparison.cpp
include/orbitalforge/util/statistics.hpp
tests/unit/test_rk4.cpp
tests/unit/test_statistics.cpp
tests/integration/test_compare_command.cpp
experiments/day5/dispatch_benchmark.cpp
```

14.5 Third integrator

Choose one:

- RK4, for general ODE accuracy and temporary-state practice;
- leapfrog, for another symplectic method.

RK4 gives more C++ pressure because it needs multiple intermediate states and vector arithmetic. It is the preferred option if time permits.

Implement it correctly before generic refactoring. Tests must compare convergence when `dt` is halved.

14.6 Compile-time versus runtime integrator exercise

Keep the production CLI using runtime polymorphism. Add an experimental templated runner:

```
template<StepIntegrator Integrator>
SimulationSummary run_simulation(
    Scenario scenario,
    Integrator integrator);
```

Compare with:

```
SimulationSummary run_simulation(
    Scenario scenario,
    std::unique_ptr<Integrator> integrator);
```

Discuss:

Question	Runtime virtual	Compile-time template
Choice known at runtime?	yes	usually no
Separate compilation easy?	yes	less so
Inlining opportunity	lower	higher
Binary size	often smaller	may grow
Extensibility without recompiling caller	better	worse
Error messages	often simpler	can be complex

Do not prematurely replace the working architecture. Learn both.

14.7 Ranges analysis pipeline

Build a comparison report:

1. run each requested integrator;
2. collect `SimulationSummary`;
3. sort by energy drift;
4. find fastest;
5. find most accurate;
6. filter failed runs;
7. print ranking.

Use ranges where readable:

```
auto valid_runs = summaries
    | std::views::filter([](const auto& s) { return s.valid; });
```

Then ask:

- Is the view lazy?
- What owns the underlying data?
- Can the view outlive the vector?
- What happens if the vector mutates?
- Is materialization needed?

14.8 Compile-time laboratory

Required exercises:

1. `static_assert(Vec3<int>{1,2,3}.x() == 1)` if integer vectors remain supported.
2. If vectors are floating-point constrained, explain why this assertion no longer compiles.
3. A `consteval` unit conversion helper.

4. A non-type template parameter:

```cpp
template<std::size_t N>
double mean(const std::array<double, N>&);
```

5. A type trait:

```cpp
static_assert(std::is_trivially_copyable_v<Vec3<double>>);
```

   Only keep it if the design actually satisfies it and the property matters.

14.9 Lambda laboratory

Write lambdas with:

- no capture;
- capture by value;
- capture by reference;
- initialized capture;
- mutable lambda;
- generic `auto` parameter.

Create one deliberate dangling-reference capture returned from a function, diagnose it, and delete it.

14.10 Tests

- RK4 or leapfrog one-step known case;
- convergence when `dt` halves;
- compare command ranks results;
- statistics work for empty and non-empty ranges;
- range view lifetime remains valid;
- constexpr checks compile;
- concept rejects an invalid fake integrator;
- runtime and compile-time runners produce equivalent results.

14.11 Interview drills

1. What is template instantiation?
2. Why are templates usually defined in headers?
3. What do concepts improve?
4. Compile-time versus runtime polymorphism.
5. `constexpr` versus `const`.
6. What is a lambda capture?
7. Why can capture by reference dangle?
8. `std::function` versus template callable.
9. Iterator versus range.
10. Lazy view lifetime.
11. When are algorithms clearer than loops?
12. What is `static_assert` for?

14.12 Acceptance gate

☐ three integrators exist;
☐ comparison command works;
☐ templates are constrained where useful;
☐ compile-time and runtime dispatch are both explained;
☐ no gratuitous template abstraction entered the core;
☐ ranking uses algorithms or ranges clearly;
☐ tag `day-5-modern-cpp` exists.

---

15. Day 6: Performance, Memory Layout, Multithreading, Synchronization

15.1 Daily mission

Measure the application, find a real bottleneck, and safely parallelize a meaningful workload. Learn to reason about memory rather than merely counting instructions.

15.2 Morning retrieval

From memory:

- draw virtual and templated integrator call paths;
- define a concept;
- explain a lazy range;
- state move semantics in one paragraph;
- list ownership types;
- identify the algorithmic bottleneck.

15.3 Concepts

Performance model

Understand:

- $O(N^2)$ force cost;
- constant factors;
- allocation cost;
- cache lines;
- spatial locality;
- temporal locality;
- branch prediction;
- data alignment;
- false sharing;
- compiler optimization;
- dead-code elimination in benchmarks;
- debug versus release behavior;
- measurement noise.

Data layout

Current array-of-structures:

```
std::vector<Body>
```

Potential structure-of-arrays:

```
struct BodyArrays {
    std::vector<double> mass;
```

```
    std::vector<Vec3d> position;
    std::vector<Vec3d> velocity;
};
```

Discuss:

- which fields the force kernel actually reads;
- wasted cache bandwidth from body names;
- ease of API use;
- serialization;
- SIMD potential;
- complexity of keeping arrays consistent.

Do not replace the model until measurements justify an experiment.

Threads

Cover:

- process versus thread;
- `std::thread`;
- `std::jthread`;
- joining;
- stop token;
- race condition;
- data race;
- happens-before intuition;
- mutex;
- `lock_guard`;
- `scoped_lock`;
- `unique_lock`;
- condition variable;
- atomic;
- memory ordering at a conceptual level;
- false sharing;
- thread-safe versus reentrant;
- exception propagation from workers.

Parallel design

Prefer independent work:

- each force worker writes to a disjoint acceleration range;
- all workers read an immutable body span;
- each parameter sweep run owns its own scenario and integrator;
- result aggregation is synchronized.

Avoid shared mutation unless necessary.

15.4 Files created

```
include/orbitalforge/app/benchmark.hpp
src/app/benchmark.cpp
include/orbitalforge/app/parameter_sweep.hpp
src/app/parameter_sweep.cpp
include/orbitalforge/physics/parallel_gravity.hpp
src/physics/parallel_gravity.cpp
tests/unit/test_parallel_gravity.cpp
tests/integration/test_parameter_sweep.cpp
experiments/day6/aos_soa.cpp
experiments/day6/race.cpp
docs/performance.md
```

15.5 Benchmark harness

Use `std::chrono::steady_clock`.

Rules:

- warm up;
- run multiple repetitions;
- report median, minimum, and perhaps percentile;
- use Release mode;
- ensure results are consumed;
- isolate I/O from compute timing;
- record compiler, flags, CPU thread count, body count, and seed;
- never compare a Debug build against Release.

Benchmarks:

```
N = 10, 50, 100, 250, 500, 1000, 2000
```

Estimate scaling by comparing time ratios when N doubles.

15.6 Parallel force kernel

Use the non-symmetric row formulation for easier parallelism:

```
worker handles i in [begin,end)
for each i:
    acceleration[i] = sum over all j != i
```

Properties:

- read-only body input;
- each output index has one owner;
- no mutex in the hot loop;
- deterministic per-index summation order;
- straightforward partitioning.

Create worker partitions based on hardware concurrency but allow explicit thread count.

Use `std::jthread` so threads join automatically.

Test equality against sequential kernel within tolerance.

Discuss why the symmetric `i < j` algorithm is harder to parallelize without atomics, locks, or thread-local reduction buffers.

15.7 Parameter sweep

Implement multiple independent simulations for several `dt` values.

Concurrency architecture:

```
work queue -> worker jthreads -> local simulation -> synchronized results
```

Use:

- `std::atomic<std::size_t>` for next job index or progress;
- `std::mutex` plus `std::scoped_lock` for result insertion;
- optional `condition_variable` in a separate lab;
- `std::stop_token` for cancellation after a failed run or user request.

Be able to explain why atomic progress does not make the entire result vector thread-safe.

15.8 Race laboratory

Write intentionally unsafe code:

```
int counter = 0;
two threads increment counter many times;
```

Run ThreadSanitizer. Fix with:

1. mutex;
2. atomic.

Compare semantics and performance. Explain why `counter++` is not atomic merely because it is one line.

Then create a race on `std::vector::push_back` from multiple threads and fix result collection.

15.9 AoS versus SoA experiment

Implement only enough to compare force-kernel input layout.

Measure:

- construction time;

- force computation time;
- memory use estimate;
- code complexity.

Write a conclusion. Keeping AoS is acceptable if the performance difference is small at project scale. Engineering maturity includes declining a refactor after measurement.

15.10 Profiling

Use available tools:

- `perf` on Linux;
- compiler optimization reports;
- `gprof` only if necessary;
- sampling profiler;
- callgrind as an optional slower tool.

Answer:

- percentage of time in force kernel;
- percentage in square root;
- allocations per simulation step;
- whether output I/O dominates small scenarios;
- speedup from threads;
- where scaling saturates.

15.11 Atomics boundary

You need interview-level understanding, not lock-free wizardry.

Know:

- atomic operation is indivisible with defined synchronization semantics;
- default sequential consistency is strongest and easiest;
- relaxed ordering can be correct for isolated statistics such as a progress counter, but use it only after explaining why;
- atomic does not make compound multi-object invariants safe;
- false sharing can make separate atomics interfere through cache lines.

15.12 Tests

- sequential and parallel accelerations agree;
- one thread matches sequential path;
- more threads do not change scientific result beyond tolerance;
- empty and single-body systems work;
- thread count zero is rejected or normalized;
- sweep results contain every requested value exactly once;
- cancellation leaves resources clean;
- TSan reports no data race;
- ASan/UBSan still pass.

15.13 Interview drills

1. Race condition versus data race.
2. Mutex versus atomic.
3. Why `jthread`?
4. What is false sharing?
5. What makes the force loop parallelizable?
6. Why is symmetric pair accumulation harder to parallelize?
7. How do you benchmark correctly?
8. Why use `steady_clock`?
9. AoS versus SoA.
10. What is cache locality?
11. Why may four threads not give four-times speedup?
12. What does ThreadSanitizer find?
13. Default atomic memory ordering.
14. Why can shared vector writes be unsafe?

15.14 Acceptance gate

☐ benchmark command works;
☐ real bottleneck is documented;
☐ safe parallel workload exists;
☐ TSan is clean;
☐ sequential and parallel results agree;
☐ AoS/SoA decision is evidence-based;
☐ speedup and saturation are reported;
☐ tag `day-6-performance` exists.

---

16. Day 7: Hardening, Interview Conversion, Release, and Independent Extension

16.1 Daily mission

Turn the repository into a defendable portfolio artifact and prove that you can extend it without step-by-step guidance.

No large new core feature is permitted before the release candidate passes all gates.

16.2 Morning closed-book challenge

In 60 minutes, on a new branch and without copying project files:

1. implement a small `Vec2<double>`;
2. implement a two-particle acceleration function;
3. write three tests;
4. explain ownership;
5. compile with warnings.

This reveals whether the week produced recall or merely familiarity.

16.3 Release checklist

Correctness

- all tests pass;
- two-body scenario remains bounded;
- energy, momentum, and center-of-mass drift are reported;
- integrator comparison behaves plausibly;
- seed reproduces generated scenario;
- parallel and sequential modes agree.

Safety

- strict warning build clean;
- ASan clean;
- UBSan clean;
- TSan clean;
- no owning raw pointer;
- no dangling view;
- no unchecked index in untrusted input path;
- no exception escapes destructors;
- no data race.

Design

- headers expose minimal dependencies;
- public interfaces express ownership;
- classes have clear responsibilities;
- inheritance exists only for strategy behavior;

- rule of zero is the default;
- `const` is pervasive where appropriate;
- names use consistent style;
- no giant `main.cpp`;
- no unnecessary global mutable state.

Product

- README build instructions work from clean clone;
- example commands work;
- scenarios included;
- output sample included;
- architecture diagram included;
- limitations stated honestly;
- performance results included;
- license chosen if repository is public.

16.4 Test pyramid

Ensure the suite includes:

Unit tests

Small pure components:

- vector arithmetic;
- parsing helpers;
- diagnostics;
- integrator steps;
- gravity.

Property tests

General invariants:

- force symmetry;
- translation invariance;
- deterministic seed;
- permutation consistency.

Integration tests

Subsystems together:

- parse scenario, simulate, write output;
- CLI exit code and files;
- comparison command;
- parallel sweep.

Regression tests

Specific bugs from the error ledger. Every serious bug earns a permanent test.

16.5 Debugger examination

Use GDB or LLDB to:

- set breakpoint in gravity kernel;
- inspect body vector;
- step into virtual integrator call;
- inspect dynamic type;
- watch a position change;
- view call stack;
- break on thrown exception;
- examine a moved-from pointer;
- inspect threads.

You should be able to debug without adding print statements.

16.6 Architecture explanation

Prepare a five-minute spoken explanation using this sequence:

1. product behavior;
2. data ownership;
3. numerical kernel;
4. integration strategy;
5. I/O and error boundary;
6. tests and scientific validation;
7. performance and concurrency;
8. one tradeoff you would revisit.

Then answer hostile but fair questions:

- Why virtual dispatch instead of a variant?
- Why not template the entire simulation?
- Why is `Body` not polymorphic?
- Why is `shared_ptr` absent?
- Why use `span`?
- Why does the hot kernel accept output storage?
- Why does the parser use a variant or exception?
- Why velocity-Verlet?
- Why not Barnes-Hut in the MVP?
- What is the dominant cost?
- What breaks if the body vector reallocates?
- Is the parallel result deterministic?

16.7 Independent extension test

Choose one extension and implement it with minimal assistance:

Preferred: collision event detection

Add detection of bodies closer than a configured threshold.

This exercises:

- optional result;
- pair search;
- event type;
- variant or enum;
- sorting by time or distance;
- tests;
- CLI output;
- design extension.

Alternative: Barnes-Hut skeleton

Implement only:

- bounding region;
- tree node;
- insertion;
- center-of-mass aggregation;
- tree validation tests.

Do not attempt full optimization unless the skeleton is correct.

Alternative: binary checkpoint

Write and load a versioned snapshot.

This exercises:

- byte representation;
- endianness awareness;
- fixed-width integers;
- serialization;
- error handling;
- compatibility.

16.8 Final interview circuit

Circuit A: language

- object lifetime;
- value categories;
- move semantics;
- const;

- templates;
- virtual dispatch;
- exceptions;
- RAII;
- smart pointers;
- undefined behavior.

Circuit B: STL

- vector complexity and invalidation;
- maps and hash maps;
- algorithms;
- iterators;
- ranges;
- optional and variant;
- string and string_view;
- span;
- chrono;
- random.

Circuit C: systems

- stack versus heap;
- alignment and padding;
- cache locality;
- false sharing;
- threads and synchronization;
- atomics;
- sanitizers;
- linking;
- build modes;
- profiling.

Circuit D: project

- numerical stability;
- complexity;
- testing invariants;
- architecture alternatives;
- performance results;
- future roadmap.

16.9 Final acceptance gate

☐ clean build from scratch;
☐ all tests and sanitizers pass;
☐ release scenario outputs are archived;
☐ README is complete;

☐ five-minute architecture explanation recorded;
☐ 30-minute mock interview completed;
☐ independent extension merged;
☐ tag `v1.0.0` and `day-7-release` exist.

---

17. Exact Repetition Plan

The following patterns must be repeated until they become automatic.

17.1 Const and reference pattern

Every function review asks:

1. Does this parameter need ownership?
2. Does it need mutation?
3. Can it be null?
4. Is it one object or a sequence?
5. How long must the borrow live?

Expected mapping:

Need	Type shape
small independent scalar	by value
required read-only object	`const T&`
required mutable object	`T&`
optional observer	`T*` or optional reference wrapper
read-only contiguous sequence	`std::span<const T>`
mutable contiguous sequence	`std::span<T>`
transfer exclusive ownership	`std::unique_ptr<T>` or value
shared lifetime	`std::shared_ptr<T>` only with justification

Apply this review to at least five functions per day.

17.2 Value semantics pattern

For every new type, ask:

- Can it be copied meaningfully?
- Should copies be cheap or merely correct?
- Can it be moved?
- Does it own a resource?
- Can rule of zero handle it?
- What is its invariant?
- Is equality meaningful?
- Is ordering meaningful?
- Can it be default-constructed into a valid state?

17.3 Container choice pattern

For every collection, ask:

- fixed or dynamic size?
- contiguous storage needed?

- stable references needed?
- ordered lookup or hash lookup?
- insertion pattern?
- iteration frequency?
- ownership?
- thread access?
- expected size?

Default to `vector` unless evidence argues otherwise.

17.4 Error handling pattern

For every failure:

- programmer bug: assertion or contract;
- invalid external input: structured parse error;
- optional absence: optional;
- multiple valid alternatives: variant;
- exceptional boundary failure: exception;
- performance hot path: avoid throwing if practical, but never sacrifice correctness silently.

17.5 Testing pattern

Every feature receives:

1. nominal test;
2. boundary test;
3. invalid-input test;
4. property or invariant;
5. regression test after any bug.

---

18. Interview-Focused Mini-Labs

These labs take 15 to 30 minutes and should be distributed across evenings or breaks.

18.1 Pointer arithmetic and arrays

Use a built-in array:

```
double values[4]{1,2,3,4};
```

Inspect:

- array-to-pointer decay;
- `sizeof(values)` in same scope;
- `sizeof` after passing to function;
- pointer arithmetic;
- one-past-end pointer;
- why dereferencing one-past-end is invalid;
- conversion to `span`.

Do not use pointer arithmetic in the main project unless justified.

18.2 Memory layout

Create:

```
struct A { char c; double d; int i; };
struct B { double d; int i; char c; };
```

Print sizes and offsets. Explain padding, alignment, and member ordering.

Then inspect `sizeof(Body)` and estimate cache impact of storing `std::string` beside numeric data.

18.3 Virtual dispatch versus variant

Implement three tiny operations using:

1. virtual base class;
2. `std::variant` and `visit`;
3. template function.

Compare readability, extensibility, and performance conceptually.

18.4 Custom iterator

Implement a simple counting iterator or iterator over every k-th element. The goal is to understand iterator protocol, not to replace ranges.

18.5 Exception safety

Create a class that acquires two resources. Force the second acquisition to fail. Verify the first is cleaned automatically.

18.6 Perfect forwarding

After move semantics are solid, implement:

```
template<class T, class... Args>
std::unique_ptr<T> make_owned(Args&&... args);
```

Compare it with `make_unique`. Learn forwarding references and `std::forward`, then delete the helper because the standard library already solved it.

18.7 Custom allocator or PMR

On the final day or later, allocate many tree nodes with `std::pmr::monotonic_buffer_resource`. Measure before and after. This is a stretch lab, not an essential MVP component.

---

19. Scientific Computing Practices

19.1 Units

For the one-week project, use normalized units where practical. Document every scenario's units. Do not silently mix SI and normalized values.

A future extension may introduce strong unit types, but this can distract from core C++ during the week.

19.2 Floating-point discipline

- use `double` by default;
- validate `std::isfinite`;
- choose tolerances relative to value scale;
- separate absolute and relative tolerance;
- avoid exact equality for computed values;
- record maximum digits when serializing;
- understand that operation ordering changes rounding;
- do not call every numerical difference a bug.

Suggested comparison:

```
abs(a-b) <= abs_tol + rel_tol * max(abs(a), abs(b))
```

19.3 Reproducibility

Every run records:

- scenario;
- configuration;
- seed;
- compiler;
- build type;
- thread count;
- timestamp;
- Git commit if practical.

A result that cannot be reproduced is a rumor wearing a CSV costume.

19.4 Validation strategy

Use several levels:

- analytical case;
- conservation laws;
- convergence under smaller time step;
- cross-integrator comparison;
- sequential/parallel equivalence;
- regression against trusted output.

19.5 Performance honesty

Never claim improvement from one run. Report:

- setup;
- repetitions;
- variability;
- absolute time;
- speedup;
- scientific error;
- algorithmic complexity;
- hardware.

A faster method that destroys energy conservation is not automatically better.

---

20. C++ “Tricks” Worth Learning and Tricks Worth Avoiding

20.1 Useful idioms

- initialize every object;
- prefer value semantics;
- use RAII for all resources;
- prefer `make_unique`;
- use `enum class`;
- use `override`;
- use `[[nodiscard]]` on important returned values;
- use `std::exchange` when implementing moves if appropriate;
- use copy-and-swap only when it genuinely simplifies exception-safe assignment;
- use `std::as_const` to force const overload selection in a lab;
- use structured bindings;
- use `std::tie` or tuple comparisons only when readable;
- use `std::erase_if`;
- use `std::clamp`;
- use `std::iota`;
- use `std::transform_reduce` only after understanding its semantics;
- reserve vector capacity when size is predictable;
- prefer algorithms that state intent;
- isolate unsafe or low-level code.

20.2 Traps

- unnecessary `shared_ptr`;
- manual `new` and `delete`;
- returning `std::move(local)`;
- storing `string_view` to temporary data;
- capturing locals by reference in escaping lambdas;
- assuming moved-from values are empty;
- using `memcpy` on non-trivially-copyable objects;
- comparing signed and unsigned blindly;
- relying on vector element addresses after growth;
- calling virtual functions from constructors or destructors without understanding dispatch;
- throwing from destructors;
- using detached threads;
- using macros where constants or templates work;
- optimizing before measuring;
- writing clever one-liners in numerical kernels;
- overloading operators with surprising meaning;
- inheritance for code reuse instead of substitutability.

20.3 Features to recognize but not prioritize this week

- multiple inheritance;
- pointer-to-member syntax;
- advanced SFINAE;
- template metaprogramming recursion;
- coroutines;
- modules;
- lock-free data structures;
- custom allocators in production;
- expression templates;
- CRTP;
- policy-based design;
- placement new;
- custom deleters beyond a lab;
- ABI engineering.

Recognizing their purpose is enough. Mastery comes later.

---

21. Daily Deliverables Summary

Day	Product increment	Main C++ mastery
1	Buildable repository, `Vec3`, `Body`, tests	compilation, headers, types, classes, operators, templates
2	Gravity and diagnostics	containers, references, spans, iterators, complexity
3	Euler and Verlet with runtime selection	RAII, lifetime, move, smart pointers, polymorphism
4	Scenario parser, CLI, CSV	strings, views, errors, variants, optional, filesystem, streams
5	Third integrator and comparison	templates, concepts, constexpr, ranges, lambdas
6	Benchmark and parallel execution	memory layout, profiling, threads, mutexes, atomics
7	Release, mock interviews, independent extension	integration, debugging, design defense, recall

22. Minimum Daily Output

A day is not complete because you spent eight hours. It is complete when it has evidence.

Each day must produce:

- one stable tagged commit;
- one new product capability;
- at least five meaningful tests;
- at least one deliberate bug diagnosis;
- one page of learning-log notes;
- one architecture or ownership decision;
- ten spoken interview answers;
- a clean strict build;
- a three-sentence retrospective.

Retrospective template:

```
## Day N Retrospective

### I can now do without help

### I still hesitate on

### The most important bug I found

### One design decision I would defend

### Tomorrow's first failing test
```

---

23. Failure Recovery Rules

23.1 If you fall behind

Do not compress quality gates. Reduce scope in this order:

1. skip GUI or visualization;
2. skip Barnes-Hut;
3. skip SoA conversion;
4. choose leapfrog instead of RK4;
5. simplify CLI syntax;
6. keep sequential force kernel but retain parallel parameter sweep.

Never cut:

- tests;
- ownership learning;
- move semantics;
- parser lifetime safety;
- sanitizers;
- concurrency race lab;
- architecture explanation.

23.2 If physics blocks you

Use normalized units and trusted initial conditions. The project is a C++ bootcamp, not an astrophysics qualifying exam.

23.3 If CMake blocks you

Reduce to one root CMake file temporarily, but preserve separate targets. Return to helper modules after the build works.

23.4 If templates become a swamp

Return to concrete `double` implementations. Generalize only after two concrete use cases or a clear correctness benefit.

23.5 If concurrency breaks determinism

Keep a correct sequential reference path. Compare every parallel result against it. Simplify work partitioning before adding locks.

---

24. Post-Week Continuation

After the seven-day sprint, continue in this order:

Week 2

- Barnes-Hut;
- stronger units;
- binary checkpoints;
- Google Benchmark;
- continuous integration;
- clang-tidy;
- coverage;
- packaging.

Week 3

- Python bindings;
- plotting;
- SIMD;
- SoA production path;
- parallel Barnes-Hut;
- memory resources.

Week 4

- modules experiment;
- plugin architecture;
- GPU backend;
- distributed parameter sweeps;
- deeper numerical methods.

The one-week repository should remain the trunk. Extensions must preserve tests and scientific validation.

---

25. Authoritative Reference Stack

Use a narrow reference stack to avoid tutorial pinball:

1. **cppreference** for exact language and standard-library behavior.
2. **C++ Core Guidelines** for design and ownership guidance.
3. **CMake official documentation** for build behavior.
4. **LLVM sanitizer documentation** for ASan, UBSan, and TSan.
5. **Compiler Explorer** for isolated language experiments.
6. One numerical methods reference for Verlet and RK4.
7. The compiler error itself, read from the first relevant line rather than the final avalanche.

Do not consume hours of videos before coding. Read just enough to formulate the next test.

---

26. Final Competency Rubric

Score each item from 0 to 3:

- **0:** cannot explain or implement;
- **1:** recognize with notes;
- **2:** implement with minor help;
- **3:** implement and explain independently.

Language and lifetime

☐ initialization and conversions;
☐ const correctness;
☐ references and pointers;
☐ object lifetime;
☐ copy and move;
☐ RAII;
☐ rule of zero/five;
☐ virtual dispatch;
☐ templates and concepts;
☐ exceptions.

Standard library

☐ vector and array;
☐ map and unordered_map tradeoffs;
☐ iterators and invalidation;
☐ algorithms and ranges;
☐ string and string_view;
☐ optional and variant;
☐ smart pointers;
☐ filesystem and streams;
☐ chrono and random;
☐ threads and synchronization.

Engineering

☐ headers and translation units;
☐ CMake targets;
☐ unit and integration tests;
☐ debugger;
☐ sanitizers;
☐ profiling;
☐ complexity;
☐ cache reasoning;
☐ architecture;
☐ Git workflow.

Target by Day 7:

- no zeroes;
- mostly twos;
- threes in ownership, vector use, RAII, const/reference design, testing, and project architecture.

---

27. The Final Standard

At the end of the week, you should be able to answer this without notes:

> *"How does one body move from a line in a scenario file to a validated C++ object, into contiguous storage, through a non-owning physics view, across a selected integrator, into a thread-safe simulation run, and finally into a reproducible CSV, while every lifetime and failure mode remains explicit?"*

If you can trace that path through types, owners, borrows, tests, numerical formulas, build targets, and runtime behavior, you have not merely touched intermediate C++. You have built the mental skeleton of a C++ engineer.

The project is the spacecraft. The real mission is learning to see lifetimes, interfaces, invariants, and costs before the compiler is forced to shout about them.